

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ
Кафедра математичного моделювання та аналізу даних**

ЗВІТ
з переддипломної практики

за спеціальністю: 113 Прикладна математика

На тему: «Колоризація зображень з інтерактивною взаємодією користувача та багатоваріантною генерацією результатів. Аналіз та порівняння різних варіантів колоризації.»

Виконав: здобувач ступеня бакалавра
4 курсу групи ФІ-21

Климентьєв Максим Андрійович

(ПІБ здобувача)

(підпис здобувача)

Керівник:
Железняков Дмитро Валентинович

(ПІБ керівника)

(підпис керівника)

Захистив з оцінкою:

Київ 2026 р.

ЗМІСТ

Індивідуальне завдання на практику	4
1 Теоретичні основи колоризації зображень	5
1.1 Поняття та історія колоризації	5
1.2 Підходи до колоризації	7
1.2.1 Згорткові нейронні мережі (CNN)	7
1.2.2 Генеративно-змагальні мережі (GAN)	7
1.2.3 Трансформери	8
1.2.4 Дифузійні моделі	8
1.2.5 Моделі простору станів (SSM)	8
1.3 Інтерактивна колоризація та багатоваріантні результати	9
1.4 Метрики якості колоризації	9
1.5 Більш детально про існуючі підходи до колоризації	11
1.5.1 Scribble-based	11
1.5.2 Example-based	12
1.5.3 Згорткові нейронні мережі (CNN)	12
1.5.4 Генеративно-змагальні мережі (GAN)	13
1.5.5 Трансформери та механізми уваги	13
1.5.6 Дифузійні імовірнісні моделі	14
1.5.7 Моделі простору станів (SSM) та архітектура Mamba	15
1.6 Порівняння різних методів колоризації	17
Висновки до розділу 1	19
2 Архітектура та методологія побудови системи керованої колоризації	20
2.1 Загальна концепція та конвеєр обробки даних системи	20
2.1.1 Стратегія симуляції підказок користувача під час навчання	21
2.1.2 Автоматизована симуляція підказок на етапі валідації	23
2.2 Компонентна деталізація та алгоритми навчання гібридної моделі ...	23
2.2.1 Згорткові блоки енкодера та декодера	24
2.2.2 Двонаправлене просторове сканування	24

2.2.3	Механізм спільного використання ваг	25 ³
2.2.4	Загальна архітектура	26
2.2.5	Формулювання функцій втрат	27
2.3	Програмна реалізація та графічний інтерфейс користувача	28
	Висновки до розділу 2	30
3	Експериментальний аналіз результатів та оцінка ефективності системи...	31
3.1	Методологія тестування та базові моделі (Baselines).....	31
3.2	Об’єктивний аналіз якості генерації та швидкодії.....	32
3.3	Оцінка перцептивної реалістичності та точності керування	33
3.4	Суб’єктивний візуальний аналіз.....	34
	Висновки до розділу 3	34
	Перелік посилань.....	35
	Додаток А Фрагменти вихідного коду програми.....	40
A.1	Реалізація двонаправленого блоку Shared Mamba	40
A.1.1	Допоміжні модулі	40
A.1.2	Основний модуль.....	41
A.2	Генерація Гауссівських підказок.....	43
A.3	Як виглядає сам даталоадер.....	46
A.4	Комбінована функція втрат (Colorization Loss)	48

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ НА ПРАКТИКУ

Темою практики є «Колоризація зображень з інтерактивною взаємодією користувача та багатоваріантною генерацією результатів. Аналіз та порівняння різних варіантів колоризації.»

Завданням є:

- 1) Зібрати та підготувати навчальну вибірку даних для тренування власної генеративної моделі колоризації зображень
- 2) Знайти та доповнити, за необхідності, датасет для єдиного тестування моделей колоризації зображень
- 3) Розробити власний алгоритм колоризації зображень на основі генеративної архітектури
- 4) Розробити програмну реалізацію системи автоматичної та інтерактивної колоризації зображень з багатоваріантним генеруванням результатів
- 5) Провести порівняння реалізованих методів за об'єктивними метриками та за суб'єктивним візуальним аналізом.

1 ТЕОРЕТИЧНІ ОСНОВИ КОЛОРИЗАЦІЇ ЗОБРАЖЕНЬ

У цьому розділі розглядаються основні теоретичні аспекти колоризації зображень, включно з історією розвитку методів, класичними алгоритмічними підходами та сучасними нейромережевими моделями. Особлива увага приділяється архітектурам від згорткових мереж до дифузійних моделей, а також методам оцінки якості отриманих результатів.

Основна мета цього розділу — проаналізувати еволюцію методів колоризації від класичних згорткових мереж до сучасних трансформерів, виявити їхні обмеження щодо обчислювальної складності та збереження просторових контурів об'єктів, і на основі цього обґрунтувати необхідність переходу до гібридних ієрархічних моделей на базі селективних просторів станів.

1.1 Поняття та історія колоризації

Колоризація — це процес додавання кольорової інформації до монохромних зображень. З математичної точки зору, цю задачу можна сформулювати як відображення одноканального сірого зображення у триканальне кольорове зображення, наприклад, у просторі RGB, CIELAB або YUV [1]. Оскільки одному значенню яскравості може відповідати безліч кольорових відтінків, задача є невизначеною та має характер «один до багатьох», що вимагає використання апріорних знань про семантику сцени.

Історично методи колоризації можна поділити на три основні етапи:

1) **Ручна та напівавтоматична колоризація:** вимагала значних зусиль художників або використання простих евристик на основі сегментації.

2) **Методи на основі глибокого навчання (CNN та GAN):** поява великих наборів даних, таких як ImageNet та COCO, дозволила тренувати згорткові мережі для передбачення кольору [2]. Знаковим став підхід *Colorful*

Image Colorization [3], де задачу було переформульовано з класичної регресії на класифікацію. Замість того, щоб мінімізувати усереднену похибку, яка призводить до тьмяних кольорів, модель передбачає розподіл ймовірностей для кожного пікселя по палітрі відтінків.

3) Генеративні та послідовні моделі (Transformers, Diffusion та SSM Models): сучасний етап, що характеризується використанням механізмів глобальної уваги та імовірнісних моделей дифузії для генерації високоякісних текстур, а також впровадженням селективних моделей простору станів (SSM) для лінійного моделювання контексту зображень та забезпечення бездоганної семантичної узгодженості [4, 5].

Розвиток методів глибокого навчання, зокрема поява архітектур U-Net та GAN, дозволив перейти від простого розфарбовування до генерації фотореалістичних зображень [6]. Останні дослідження все частіше фокусуються на дифузійних моделях, які демонструють state-of-the-art результати у задачах синтезу зображень [7].

Поряд із цим, активного розвитку набули архітектури на базі механізмів глобальної уваги. Проте висока обчислювальна складність класичних трансформерів та нездатність їхніх лінійних альтернатив зберігати точний глобальний контекст роблять їх неоптимальними для інтерактивних задач.

Водночас новітні архітектури на базі SSM здатні забезпечити глобальне рецептивне поле з лінійною складністю. Хоча вони й почали з'являтися у вузькоспеціалізованих задачах відновлення інфрачервоних [8, 9] та супутникових знімків [10], вони залишаються недослідженими в контексті інтерактивної колоризації фотографій. Відсутність рішень, які б поєднували лінійну обчислювальну ефективність просторів станів із точним просторовим контролем за допомогою користувацьких підказок, створює простір для наукового пошуку та підкреслює актуальність запропонованого в цій роботі гібридного підходу.

1.2 Підходи до колоризації

Підходи до колоризації базуються на різних архітектурах нейронних мереж. Опустимо гібридні архітектури та розглянемо деякі з базових.

1.2.1 Згорткові нейронні мережі (CNN)

Ранні підходи використовували автоенкодери для прямого передбачення каналів кольоровості (наприклад, a та b у просторі CIE Lab).

Ларссон та ін. [11, 3] запропонували використовувати ознаки, вивчені для класифікації об'єктів, для задачі колоризації, демонструючи тісний зв'язок між семантикою об'єкта та його кольором.

Інший популярний метод, *Deep Koalarization* [12], використовує Inception-ResNet-v2 для вилучення ознак та подвійний декодер для відновлення кольору.

1.2.2 Генеративно-змагальні мережі (GAN)

З метою усунення ефекту недостатньої насиченості кольорів були запропоновані різні GAN.

- Модель *Pix2Pix* [6] використовує умовний GAN (cGAN) для перетворення зображень, що дозволяє генерувати більш чіткі та реалістичні результати.
- Метод *ChromaGAN* [13] поєднує геометричну інформацію з семантичним розподілом класів для покращення правдоподібності розфарбовування.
- Також існують підходи для непарного навчання, такі як CycleGAN [14], що дозволяють тренувати моделі без наявності пар «чорно-біле – кольорове».

1.2.3 Трансформери

З появою механізму уваги дослідники почали застосовувати трансформери для колоризації.

Наприклад, *Colorization Transformer* [15] та *UniColor* [16] використовують архітектуру трансформерів для моделювання довгих залежностей у зображенні, що покращує узгодженість кольорів на великих відстанях.

Як ще один приклад: *DDColor* [17] використовує подвійні декодери та запити кольору (color queries) для досягнення фотореалістичності.

1.2.4 Дифузійні моделі

Найбільш перспективним напрямком на сьогодні є імовірнісні дифузійні моделі. Вони працюють шляхом поступового видалення шуму з випадкового сигналу, керуючись вхідним чорно-білим зображенням.

- **Palette** [7] — універсальний фреймворк для перетворення зображень, який перевершує GAN та регресійні моделі без необхідності налаштування під конкретну задачу.

- **Cold Diffusion** [18] пропонує підхід до інверсії довільних перетворень зображень без використання гаусового шуму, що відкриває нові можливості для відновлення зображень.

1.2.5 Моделі простору станів (SSM)

Новітнім трендом у глибокому навчанні є впровадження селективних моделей простору станів, зокрема архітектури Mamba [5]. Вони розроблені як ефективна альтернатива трансформерам, що здатна моделювати наскрізні глобальні контекстні зв'язки зображення, але має лінійну обчислювальну складність, що критично для систем реального часу.

1.3 Інтерактивна колоризація та багатоваріантні результати

Оскільки колоризація є відносно суб'єктивною задачею, важливу роль відіграють методи, що дозволяють користувачеві впливати на результат.

Колоризація на основі прикладів (Exemplar-based): Методи, такі як *Deep Exemplar-based Colorization* [19], використовують референсне кольорове зображення для перенесення колірної палітри на цільове чорно-біле.

Колоризація на основі тексту (Language-based): З розвитком мультимодальних моделей, таких як CLIP, з'явилася можливість керувати кольором за допомогою текстових описів.

- **L-CAD** [20] використовує дифузійні пріори для колоризації на основі описів будь-якого рівня деталізації.

- **Diffusing Colors** [21] пропонує фреймворк, де текстові підказки керують процесом дифузії, забезпечуючи баланс між автоматизацією та контролем.

Колоризація на основі підказок користувача (User-guided): Методи *Real-Time User-Guided Image Colorization* [22] та *iColoriT* [23] дозволяють користувачеві ставити кольорові точки або штрихи, які мережа поширює на відповідні семантичні регіони, використовуючи, наприклад, Vision Transformer для кращого розуміння контексту.

Сучасна тенденція, як показано в *Control Color* [24] та *UniColor* [16], полягає у створенні уніфікованих фреймворків, що підтримують різні типи умов, такі як текст, зразок, точки, штрихи, в одній моделі.

1.4 Метрики якості колоризації

Оцінка якості колоризації є складною задачею через відсутність єдиного «правильного» розв'язку: одному чорно-білому зображенню може відповідати безліч правдоподібних кольорових варіацій. Тому для комплексного оцінювання розроблених моделей використовують набір різнопланових метрик, які охоплюють як математичну точність, так і перцептивний реалізм та

обчислювальну ефективність.

Використовуються наступні основні групи метрик:

- **Метрики цільової похибки (MSE або MAE):** У задачах інтерактивної колоризації середньоквадратичну або середньоабсолютну похибку доцільно розділяти на дві складові:

- MSE_{LAB}/MAE_{LAB} — загальна похибка відновлення кольору в усьому просторі зображення.

- MSE_{HINT} — похибка виключно в тих точках, де користувач надав підказки. Ця метрика є критично важливою для керованих моделей, оскільки значення, близькі до нуля, доводять ідеальну інтеграцію та безумовне дотримання моделлю користувацького вводу без ефекту ігнорування.

- **Піксельні та структурні метрики:** PSNR (Peak Signal-to-Noise Ratio) та SSIM (Structural Similarity Index). Вони попіксельно порівнюють згенерований результат з оригінальним кольоровим зображенням. Хоча вищі значення вказують на краще відновлення оригіналу, ці метрики часто не корелюють з людським сприйняттям, оскільки «інший» правдоподібний колір математично розцінюється як помилка.

- **Перцептивні генеративні метрики:** LPIPS (Learned Perceptual Image Patch Similarity), FID (Fréchet Inception Distance) та KID (Kernel Inception Distance). На відміну від піксельних метрик, вони порівнюють не конкретні пікселі, а високорівневі семантичні ознаки, вилучені за допомогою попередньо навчених мереж, таких як VGG чи Inception. Зниження показників FID та KID свідчить про те, що розподіл згенерованих кольорів максимально наближений до розподілу реальних фотографій, що підтверджує високу візуальну реалістичність. Оскільки метрика KID обчислюється ітеративно на випадкових підмножинах даних, у роботі вона подається у форматі двох значень:

- KID_{mean} — середнє значення метрики, визначає загальну якість генерації.

- KID_{std} — стандартне відхилення, визначає стабільність та дисперсію результатів моделі на різних вибірках.

- **Обчислювальна та апаратна ефективність:** Для систем, що

передбачають інтерактивну роботу користувача, фундаментальне значення мають показники швидкодії. До них належать час інференсу (Time, вимірюється в мілісекундах) та обсяг споживаної відеопам'яті (VRAM). Сучасні архітектури повинні забезпечувати баланс між високою генеративною якістю та мінімальним споживанням ресурсів графічного процесора.

1.5 Більш детально про існуючі підходи до колоризації

До епохи глибокого навчання колоризація розглядалася переважно як задача оптимізації. Класичні алгоритмічні методи можна розділити на дві великі групи: методи на основі втручання користувача (Scribble-based) та методи перенесення кольору зі зразку (Example-based).

1.5.1 Scribble-based

Цей підхід базується на припущенні, що сусідні пікселі з подібною інтенсивністю повинні мати подібний колір. Користувач наносить на чорно-біле зображення кольорові штрихи, які слугують граничними умовами. Задача зводиться до мінімізації цільової функції енергії J [1, 25]:

$$J(U) = \sum_r \left(U(r) - \sum_{s \in N(r)} w_{rs} U(s) \right)^2,$$

де $U(r)$ — колір у пікселі r , $N(r)$ — окіл пікселя, а w_{rs} — ваговий коефіцієнт, що залежить від подібності яскравості пікселів r та s .

Перевагою методу є повний контроль користувача, проте він вимагає значних ручних зусиль і часто дає артефакти на межах об'єктів зі слабким контрастом.

1.5.2 Example-based

У цьому підході колір переноситься з референсного кольорового зображення на цільове чорно-біле шляхом зіставлення статистик яскравості та текстур.

Ранні роботи [26, 27] використовували зіставлення гістограм у декорельованому колірному просторі $l\alpha\beta$ (простір Рудермана), що дозволяло маніпулювати колірними каналами незалежно один від одного без виникнення візуальних артефактів.

Хоча цей метод є більш автоматизованим, він критично залежить від вдалого вибору референсного зображення та часто призводить до помилкового забарвлення семантично різних об'єктів, що мають схожу текстуру.

1.5.3 Згорткові нейронні мережі (CNN)

Поява CNN дозволила моделювати складні залежності між формою об'єкта та його ймовірним кольором, використовуючи великі набори даних.

Перші нейромережеві методи намагалися прямо передбачити значення каналів ab у просторі Lab, мінімізуючи середньоквадратичну помилку (MSE):

$$\mathcal{L}_{L2} = \frac{1}{2} \sum_{h,w} \|Y_{gt} - \hat{Y}\|_2^2,$$

де Y_{gt} — справжнє зображення, а $\hat{Y}$ — згенероване.

Проте, як зазначають Zhang et al. [3], використання MSE призводить до усереднення всіх можливих кольорів, що дає ненасичені результати.

Для вирішення цієї проблеми задачу було переформульовано як мультимодальну класифікацію, де мережа передбачає розподіл ймовірностей для квантованих відтінків кольору, що значно підвищило яскравість результатів.

1.5.4 Генеративно-змагальні мережі (GAN)

Мережі GAN, зокрема архітектура Pix2Pix [6], здійснили прорив у чіткості зображень. Генератор G намагається створити реалістичне зображення, щоб обдурити дискримінатор D , який вчиться відрізняти справжні кольорові фото від згенерованих. Функція втрат GAN має вигляд:

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))].$$

Підходи на кшталт ChromaGAN [13] інтегрують семантичну інформацію у процес генерації, що дозволяє уникнути грубих помилок, наприклад, зеленого неба.

1.5.5 Трансформери та механізми уваги

Останні дослідження [15, 16, 17] показують, що класичні згорткові мережі мають локальне рецептивне поле, через що вони погано справляються з моделюванням глобальних зв'язків. Це часто призводить до семантичних помилок: наприклад, різні частини одного об'єкта (як-от капот і дах автомобіля), перекриті іншими елементами сцени, можуть бути розфарбовані в різні кольори.

Перехід до архітектур на базі трансформерів дозволив вирішити цю проблему. Вони використовують механізм глобальної уваги (Self-Attention), який дозволяє кожному патчу зображення взаємодіяти з усіма іншими патчами одночасно, враховуючи контекст усієї сцени. Це є критично важливим для просторово узгодженого забарвлення великих областей та підтримки єдиної логіки освітлення.

Зокрема, модель Colorization Transformer [15] вперше успішно застосувала цей підхід, розбивши зображення на послідовність патчів і розглядаючи задачу колоризації як задачу прогнозування послідовності токенів (подібно до генерації тексту). Модель генерує кольори авторегресійно, що забезпечує високу якість, проте робить процес інференсу надзвичайно повільним.

Значним кроком у напрямку інтерактивності стала модель UniColor [16]. Вона запропонувала уніфікований фреймворк, який здатний обробляти мультимодальні умови — від автоматичної колоризації до генерації на основі текстових описів, референсних зображень або точкових підказок (штрихів). Використання трансформера дозволило ефективно інтегрувати ці різні підказки у єдиний простір репрезентацій.

Для вирішення проблеми розмиття контурів, яка часто виникає у чистих трансформерів, архітектура DDColor [17] впровадила систему подвійних декодерів (Dual Decoders). Один декодер відповідає виключно за відновлення просторової роздільної здатності (піксельний рівень), а інший — за формування кольорових семантичних ознак (глобальний контекст), після чого їхні результати зливаються. Це дозволило досягти високого рівня фотореалістичності.

Проте, незважаючи на беззаперечні переваги в якості генерації, використання класичного механізму уваги накладає жорсткі апаратні обмеження. Обчислювальна складність Self-Attention зростає квадратично $\mathcal{O}(N^2)$ зі збільшенням роздільної здатності зображення. Це робить такі моделі вкрай вимогливими до обсягу відеопам'яті (VRAM) і ускладнює їх використання для інтерактивної колоризації зображень у високій якості в реальному часі.

Спроби подолати цю проблему за рахунок так званих «ефективних трансформерів» призводять до нових компромісів. Наприклад, обчислення уваги в локальних вікнах, наприклад, в Swin Transformer [28], або використання математичних лінійних апроксимацій змушують модель втрачати здатність до безкомпромісного та миттєвого глобального моделювання контексту [29, 30]. У задачі інтерактивної колоризації це стає критичним недоліком, оскільки неймережа втрачає здатність швидко та безшовно поширювати колір від точкової підказки користувача на віддалені ділянки об'єкта.

1.5.6 Дифузійні імовірнісні моделі

На відміну від GAN, які генерують результат за один прохід, дифузійні імовірнісні моделі формують зображення ітеративно. Їхня математична основа

базується на двох марковських процесах: прямому, який поступово руйнує дані шляхом додавання гаусового шуму, та зворотному, де нейронна мережа вчиться крок за кроком відновлювати оригінальний сигнал, керуючись вхідним чорно-білим зображенням як просторовою умовою.

Цей клас моделей здійснив революцію в синтезі зображень завдяки високій стабільності навчання, тобто відсутності проблеми колапсу мод, який характерний для GAN, та неперевершеній якості генерації дрібних текстур. Знаковою роботою в цій галузі став універсальний фреймворк Palette [7], який продемонстрував, що умовна дифузійна модель здатна перевершити спеціалізовані GAN-архітектури у задачі колоризації без додаткового налаштування під специфіку предметної області.

Поряд із класичним підходом активно розвиваються альтернативні формулювання генеративного процесу. Наприклад, метод Cold Diffusion [18] довів можливість інверсії довільних детермінованих перетворень зображень без використання чистого гаусового шуму, що розширює теоретичні межі застосування дифузійних алгоритмів.

Проте фундаментальним недоліком дифузійних моделей залишається їхня надзвичайно висока обчислювальна вартість на етапі інференсу. Оскільки для створення фінального зображення модель повинна виконати десятки або сотні послідовних ітерацій денойзингу, час генерації зростає на порядки порівняно з моделями прямого проходу. У контексті керованої інтерактивної колоризації, де фундаментальною вимогою є миттєвий візуальний зворотний зв'язок на кожну нову підказку користувача, використання чистих дифузійних моделей є недоцільним через руйнування користувацького досвіду.

1.5.7 Моделі простору станів (SSM) та архітектура Mamba

Новітнім класом нейромережових архітектур, що дозволяє подолати квадратичну складність трансформерів та повільність дифузійних моделей, є селективні SSM, зокрема архітектура Mamba [5]. Вони здатні моделювати наскрізні глобальні контекстні зв'язки з суворою лінійною обчислювальною

складністю $\mathcal{O}(N)$, що робить їх ідеальними для систем реального часу.

Математично базова система SSM відображає вхідний одновимірний сигнал $x(t) \in \mathbb{R}^L$ у прихований стан $h(t) \in \mathbb{R}^N$ через лінійні диференціальні рівняння:

$$\begin{aligned} h'(t) &= A \cdot h(t) + B \cdot x(t), \\ y(t) &= C \cdot h(t) + D \cdot x(t), \end{aligned}$$

де A, B, C, D — матриці параметрів системи. При цьому матриця A характеризує внутрішню динаміку системи та керує еволюцією прихованого стану, тобто відповідає за довготривалу пам'ять, матриця D описує прямий зв'язок між входом і виходом, виконуючи роль класичного пропускового з'єднання, а матриці B та C визначають коефіцієнти взаємодії прихованого стану із вхідним та вихідним сигналами відповідно.

Для обробки дискретних послідовностей безперервні параметри трансформують у дискретні $(\bar{A}, \bar{B})$ із використанням кроку дискретизації Δ . Зазвичай для цього застосовують метод фіксації нульового порядку, Zero-Order Hold:

$$\begin{aligned} \bar{A} &= \exp(\Delta A), \\ \bar{B} &= (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B, \end{aligned}$$

що дозволяє переписати систему у дискретному вигляді: $h_k = \bar{A} \cdot h_{k-1} + \bar{B} \cdot x_k$, $y_k = C \cdot h_k + D \cdot x_k$.

Головна інновація архітектури Mamba полягає у селективності її внутрішніх механізмів: параметри B, C та крок дискретизації Δ стають динамічними функціями від вхідного сигналу, на відміну від статичних матриць A та D . Завдяки такій дискретизації параметрів та використанню апаратно-оптимізованого алгоритму паралельного сканування в пам'яті графічного процесора, модель динамічно фільтрує нерелевантну інформацію та ефективно обробляє довгі послідовності без обчислювальних обмежень механізму уваги.

Однак пряме застосування оригінальної одновимірної Mamba до

двовимірних зображень призводить до проблеми втрати локального контексту, оскільки розгортання матриці пікселів у вектор розриває їхню просторову суміжність по вертикалі. Для вирішення цієї проблеми візуальні адаптації на кшталт Vision Mamba [31] та новітні моделі спектральної трансляції [8] використовують механізми двонаправленого або перехресного сканування. Це дозволяє моделі агрегувати ознаки з різних напрямків, повністю відновлюючи просторове розуміння сцени.

У контексті керованої колоризації використання Mamba відкриває унікальну можливість: здатність моделі миттєво та узгоджено поширювати колірну інформацію від точкової підказки користувача на весь об'єкт завдяки глобальному рецептивному полю, уникаючи при цьому експоненційного зростання споживання відеопам'яті. Саме цей синергетичний баланс обчислювальної ефективності та просторової експресивності робить SSM найбільш перспективним фундаментом для розробки сучасних інтерактивних систем розфарбовування.

1.6 Порівняння різних методів колоризації

Узагальнимо характеристики розглянутих методів у порівняльній таблиці

1.1. Для аналізу обрано такі ключові критерії:

- **Якість (Quality):** рівень фотореалістичності, природність та насиченість згенерованих кольорових відтінків.
- **Узгодженість (Consistency):** просторова зв'язність кольору в межах об'єктів та відсутність візуальних артефактів.
- **Керованість (Control):** здатність моделі точно інтерпретувати та безумовно інтегрувати підказки користувача.
- **Швидкість (Speed):** обчислювальна складність та час інференсу, що визначають придатність до роботи в реальному часі.

Проведений аналіз демонструє наявність компромісу між обчислювальною лінійністю та збереженням локальних просторових контурів у сучасних послідовних моделях. Чисті трансформери забезпечують високу якість

Таблиця 1.1 – Порівняльний аналіз методів колоризації зображень

Метод / Архітектура	Представники	Переваги	Недоліки
Класичні оптимізаційні	Levin et al. [25], Welsh [27]	Точний локальний контроль користувача, відсутність семантичних «галюцинацій»	Обчислювально повільні, вимагають значної ручної розмітки, пасують перед складними текстурами
CNN	Zhang et al. [3], Larsson [11]	Висока швидкість інференсу, стабільність процесу навчання	Обмежене рецептивне поле; схильність до генерації усереднених, тьмяних відтінків при мінімізації MSE
GAN	Pix2Pix [6], ChromaGAN [13]	Висока чіткість контурів, насичені та візуально привабливі кольори	Нестабільність змагального навчання, схильність до появи кольорних артефактів та плям
Transformers	ColTran [15], UniColor [16], DDColor [17]	Моделювання глобального контексту всієї сцени, гнучка мультимодальна керованість	Квадратична обчислювальна складність $\mathcal{O}(N^2)$, критично високі вимоги до обсягу відеопам'яті (VRAM)
Diffusion Models	Palette [7], Cold Diffusion [18]	Найвища якість генерації (SOTA), бездоганий фотореалізм текстур	Екстремально низька швидкість інференсу через сотні послідовних ітерацій денойзингу
Mamba SSM	Vision Mamba [31], ColorMamba [8]	Суворо лінійна складність $\mathcal{O}(N)$, стабільне споживання VRAM, глобальне рецептивне поле	Чутливість до одновимірного розгортання 2D-даних, ризик втрати або розмиття локальних контурів об'єктів

зв'язності, але стають апаратно непридатними для обробки великих зображень у реальному часі, тоді як SSM пропонують бажану лінійну ефективність $\mathcal{O}(N)$, але потребують додаткових механізмів локального посилення контурів.

Таким чином, гібридні підходи, що поєднують генеративну силу та швидкість селективних лінійних моделей простору станів із точністю локального вилучення ознак за допомогою ієрархічних згорткових декодерів, виглядають найбільш обґрунтованими та перспективними для вирішення задачі керованої інтерактивної колоризації. Саме розробці та дослідженню такої гібридної архітектури присвячено наступні розділи цієї роботи.

Висновки до розділу 1

У цьому розділі було проведено огляд теоретичних основ та сучасних методів колоризації зображень. Було показано, що еволюція методів пройшла шлях від простих евристик до складних генеративних моделей, таких як GAN, Transformers, дифузійні фреймворки та селективні моделі простору станів.

Аналіз літератури свідчить, що в той час як задача безумовної колоризації досягла значних успіхів у реалізмі, існують відкриті питання в області обчислювальної складності моделей при роботі з високою роздільною здатністю, керованості процесом та багатоваріантності для усіх моделей. Зокрема, гібридні підходи, мультимодальні підходи та уніфіковані дифузійні й рекурентні структури є найбільш актуальним напрямком досліджень.

Порівняння нейромережових підходів показало, що хоча архітектури GAN забезпечують швидкий інференс, вони часто страждають від нестабільності навчання та появи кольірних артефактів. Дифузійні моделі дозволяють досягти найвищої якості та фотореалізму текстур, проте їхнє практичне застосування жорстко обмежене тривалим часом інференсу через ітеративний характер процесу генерації. У свою чергу, підходи на базі трансформерів ефективно моделюють глобальний контекст усієї сцени, однак квадратичне зростання обчислювальних витрат $\mathcal{O}(N^2)$ суттєво ускладнює їх масштабування для зображень високої роздільної здатності.

У наступному розділі буде запропоновано власний підхід до вирішення задачі колоризації зображень, що базується на гібридизації архітектури U-Net та Mamba SSM з використанням спільних ваг. Це дозволить ефективно поєднати вилучення низькорівневих просторових ознак, для боротьби з color bleeding, із лінійним моделюванням глобального контексту та суттєвим збереженням пам'яті графічного процесора.

2 АРХІТЕКТУРА ТА МЕТОДОЛОГІЯ ПОБУДОВИ СИСТЕМИ КЕРОВАНОЇ КОЛОРИЗАЦІЇ

У цьому розділі представлено опис спроектованої системи інтерактивної колоризації зображень, що базується на інтеграції SSM та U-Net архітектур. Розглядається методологія підготовки даних, принципи формування та кодування користувацьких підказок, а також загальний конвеєр обробки інформації.

Окрему увагу приділено компонентній деталізації розробленої гібридної моделі, математичному обґрунтуванню механізмів двонаправленого сканування та оптимізації функцій втрат для забезпечення стабільного навчання.

2.1 Загальна концепція та конвеєр обробки даних системи

Розроблена система інтерактивної колоризації функціонує в межах конвеєра обробки на основі підказок, який поєднує автоматичне вилучення семантичних ознак монохромного зображення із просторовим керуванням через визначені користувачем точкові орієнтири.

На відміну від повністю автоматичних методів, які розв'язують задачу «один до багатьох» виключно на основі апріорних знань нейромережі, запропонований підхід дозволяє задавати цільові кольори для конкретних семантичних областей зображення, трансформуючи задачу у стохастичну генерацію з граничними умовами.

Математично вхідний потік даних формується шляхом злиття інформації з двох різних модальностей:

1) **Канал яскравості:** Одноканальне чорно-біле зображення $I_L \in \mathbb{R}^{1 \times H \times W}$, вилучене як компонент L кольорового простору CIELAB та нормалізоване до $[-1, 1]$.

2) **Тензор підказок (Hints):** Триканальний просторовий масив

$I_{\text{hints}} \in \mathbb{R}^{3 \times H \times W}$, який кодує хроматичну інформацію у точках взаємодії користувача з інтерфейсом.

Для того, щоб нейромережа могла чітко диференціювати точки, де користувач свідомо задав колір, від порожнього простору, тензор підказок I_{hints} розділяється на:

- Канали кольоровості a та b , розмірності $2 \times H \times W$, нормалізовані у $[-1, 1]$, які містять цільові хроматичні значення у координатах заданих точок, а у решті пікселів заповнені нулями.
- Маску інтенсивності підказок $M \in [0, 1]^{1 \times H \times W}$, де значення більше певного шуму, близького до 0, наприклад 0.05, є користувацькою підказкою, а менше певного шуму вказує на відсутність в цьому місці підказки. Без використання цього каналу модель сприймала б нулі у порожніх зонах як вимогу розфарбувати всю картинку у нейтральний сіро-зелений колір.

Перед подачею в енкодер канали об'єднуються вздовж виміру каналів за допомогою операції конкатенації, утворюючи уніфікований чотириканальний тензор $X_{\text{in}} = \text{Concat}(I_L, I_{\text{hints}}) \in \mathbb{R}^{4 \times H \times W}$.

2.1.1 Стратегія симуляції підказок користувача під час навчання

Оскільки під час реального використання кількість, розмір та розташування точок є випадковими, процес навчання моделі вимагає динамічної симуляції дій користувача на кожній ітерації. Навчання на фіксованих шаблонах призвело б до перенавчання моделі та ігнорування підказок на інференсі.

Конвеєр підготовки даних під час тренування реалізує таку стохастичну процедуру для кожного батчу:

1) З оригінального кольорового зображення вилучаються істинні канали кольоровості $Y \in \mathbb{R}^{2 \times H \times W}$.

2) Випадковим чином визначається цільова кількість підказок K з дискретного геометричного розподілу $K \sim \text{Geom}(p = 0.2)$, в якому математичне сподівання становить $\mathbb{E}[K] = \frac{1-p}{p}$, тобто в середньому генерується

4 підказки. Ймовірність появи конкретної кількості підказок k обчислюється як $P(K = k) = p(1 - p)^k$. Зокрема, ймовірність генерації зображення взагалі без підказок, тобто $K = 0$, становить 20%, що змушує модель також вчитися безумовній автоматичній колоризації.

3) Для імітації фокусу людської уваги координати підказок генеруються за допомогою алгоритму виділення візуальної значущості. Алгоритм обчислює двовимірну функцію щільності ймовірності (PDF) на основі каналу яскравості I_L . Спочатку за допомогою згортки з операторами Собеля (S_x, S_y) обчислюється магнітуда градієнтів:

$$G = \sqrt{(I_L * S_x)^2 + (I_L * S_y)^2} + \epsilon$$

Для поширення значущості від контурів усередину об'єктів до матриці G застосовується просторове усереднення. Отримана карта нормалізується та змішується з рівномірним розподілом U з ваговим коефіцієнтом $\alpha = 0.15$, щоб забезпечити ненульову ймовірність вибору точок навіть в абсолютно однорідних областях зображення:

$$\text{PDF} = (1 - \alpha) \frac{G_{\text{blur}}}{\sum G_{\text{blur}}} + \alpha U$$

Координати семплюються з цього розподілу без повернення. Додатково застосовується евристика мінімальної просторової відстані, яка відкидає нові координати, якщо вони знаходяться занадто близько до вже існуючих, уникаючи неприродного скупчення кліків.

4) Формується маска інтенсивності M : у вибраних координатах розміщуються двовимірні радіальні гаусові ядра випадкового радіуса та випадкової амплітуди інтенсивності (від 0.2 до 1.0). Це імітує різні розміри пензля користувача та ступінь «впевненості» підказки. Якщо гаусові плями перекриваються, маска оновлюється за принципом максимуму $M_{\text{new}} = \max(M_{\text{old}}, G_{\text{patch}})$.

5) Формуються канали підказок a та b шляхом поелементного множення істинних значень Y на згенеровану маску M .

2.1.2 Автоматизована симуляція підказок на етапі валідації

Для об'єктивної оцінки якості моделі під час валідації та тестування, алгоритм перемикається з випадкової генерації у детермінований «експертний» режим. Алгоритм автоматично знаходить найбільш інформативні зони для колоризації, діючи за такою логікою:

1) Обчислюється матриця спектральної магнітуди, насиченості кольору, для всього зображення як сума квадратів значень каналів a та b .

2) Визначаються координати пікселя з максимальною магнітудою, і в цій точці розміщується гаусова підказка фіксованого розміру.

3) Щоб запобігти вибору точок в одному й тому самому об'єкті, застосовується механізм просторового пригнічення: значення магнітуди в заданому радіусі навколо обраної точки примусово встановлюються у від'ємні значення, а саме -1.0 .

4) Процес ітеративно повторюється для заданої кількості підказок, наприклад, $K = 3$.

Такий конвеєр валідації гарантує, що модель отримує підказки саме в найскладніших, найбільш яскравих ділянках зображення, що дозволяє коректно оцінити її здатність до поширення кольору на великі просторові відстані.

2.2 Компонентна деталізація та алгоритми навчання гібридної моделі

Розроблена гібридна модель базується на ієрархічній архітектурі типу U-Net, у якій класичний центральний згортковий вузол (bottleneck) замінено на Mamba.

Такий підхід дозволяє делегувати вилучення локальних просторових ознак та збереження контурів перевіреним згортковим шарам, тоді як завдання глобального поширення кольору від точкових підказок виконується механізмами SSM з лінійною обчислювальною складністю.

2.2.1 Згорткові блоки енкодера та декодера

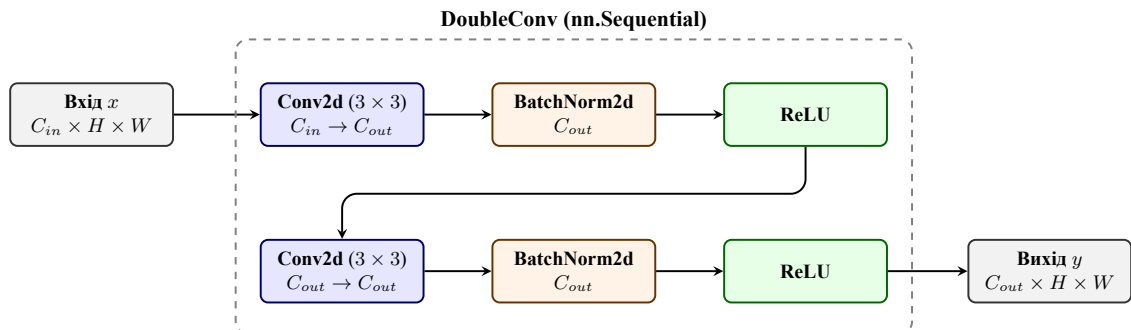


Рисунок 2.1 – Архітектура блоку DoubleConv для енкодера та декодера

Для вилучення просторових ознак та поступового зниження роздільної здатності використовується згортковий енкодер. Кожен його рівень побудований за допомогою *DoubleConv*, структурну схему якої зображено на рис. 2.1.

Блок складається з двох послідовних згорткових шарів (3×3), за кожним з яких слідує нормалізація батчів та нелінійна функція активації ReLU. Після кожного блоку в енкодері застосовується операція просторового стиснення — Max Pooling 2×2 , яка зменшує розмірність карт ознак удвічі.

Декодер працює симетрично, використовуючи білінійну інтерполяцію для масштабування.

Критичним елементом архітектури є наскрізні з'єднання, які прокидають карти ознак з енкодера безпосередньо у відповідні шари декодера, що запобігає втраті дрібних контурів об'єктів, тобто ефекту «color bleeding», під час глобальних трансформацій.

2.2.2 Двонаправлене просторове сканування

Оригінальна архітектура Mamba розроблена для обробки одновимірних послідовностей, наприклад, тексту, і сканує дані лише в одному напрямку. Для двовимірних зображень це створює проблему просторової асиметрії: пікселі не

отримують контекстної інформації від тих областей, що знаходяться нижче або правіше від них по сітці сканування.

Для вирішення цієї проблеми у bottleneck реалізовано алгоритм двонаправленого сканування. Розгорнута у вектор послідовність просторових ознак $X \in \mathbb{R}^{B \times L \times D}$, де L — це H помножена на W та поділена на певний коефіцієнт, який залежить від U-Net, а C — кількість початкових каналів, у нашому випадку це 4, обробляється паралельно: у природному напрямку та в інвертованому.

Інверсія досягається за допомогою безвитратної операції перегортання тензора у пам'яті (`torch.flip`). Фінальний вихід Y обчислюється як середнє арифметичне двох проходів:

$$Y = \frac{1}{2} (\text{Mamba}_{\text{fwd}}(X) + \text{Flip}(\text{Mamba}_{\text{rev}}(\text{Flip}(X))))$$

Це забезпечує кожному просторовому токеноу доступ до повного глобального контексту сцени з усіх напрямків.

2.2.3 Механізм спільного використання ваг

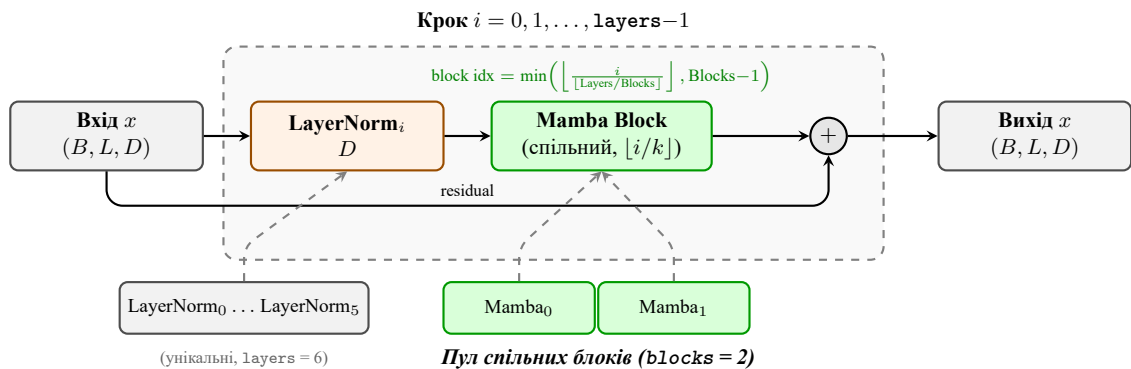


Рисунок 2.2 – Внутрішня архітектура та механізм розділення ваг у блоці Shared Mamba

Використання глибоких послідовностей Mamba-блоків вимагає значного обсягу відеопам'яті графічного процесора. Для оптимізації апаратного

споживання без втрати генеративної здатності мережі імплементовано механізм спільного використання ваг, структурну логіку якого зображено на рис. 2.2.

Замість ініціалізації L унікальних блоків Mamba, модель створює лише обмежений фізичний пул з K блоків (де $K < L$). Під час прямого проходу шари послідовно перевикористовують ці фізичні блоки згідно з математичним індексом:

$$\text{idx} = \min \left(\left\lfloor \frac{i}{\lfloor L/K \rfloor} \right\rfloor, K - 1 \right), \quad \text{де } i \in [0, L - 1]$$

Водночас, шари пакетної нормалізації (LayerNorm), які передують кожному блоку, залишаються унікальними для кожного кроку i . Такий підхід дозволяє моделі проєктувати ознаки в різні підпростори та успішно адаптуватися до різних рівнів абстракції попри спільні вагові матриці A, B, C, D всередині самих блоків простору станів.

2.2.4 Загальна архітектура

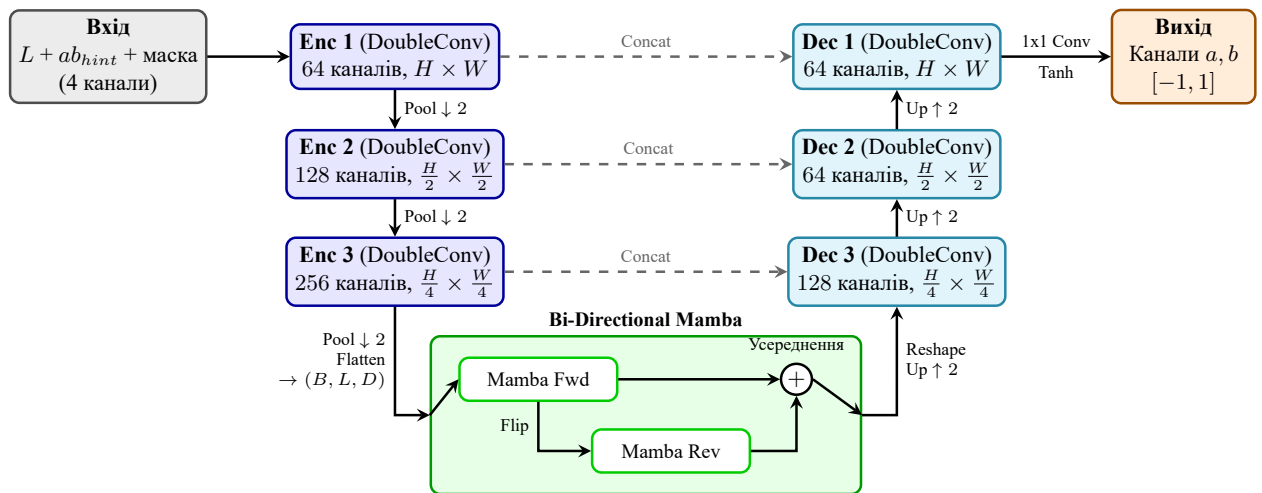


Рисунок 2.3 – Архітектура гібридної нейронної мережі Bidirectional Shared Mamba

Загальна архітектура розробленої мережі представлена на рис. 2.3.

2.2.5 Формулювання функцій втрат

Процес оптимізації гібридної мережі керується складеною цільовою функцією втрат, яка балансує піксельну точність відтворення кольору, суворе дотримання підказок користувача та візуальний, перцептивний, реалізм згенерованого зображення.

Загальна функція втрат $\mathcal{L}_{\text{total}}$ формується як зважена сума трьох незалежних компонентів:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{L1}} \mathcal{L}_{\text{L1}} + \lambda_{\text{hint}} \mathcal{L}_{\text{hint}} + \lambda_{\text{LPIPS}} \mathcal{L}_{\text{LPIPS}}$$

де λ_{L1} , λ_{hint} , λ_{LPIPS} — відповідні вагові коефіцієнти, що задають пріоритет кожної метрики.

1) **Глобальна похибка кольоровості ($\mathcal{L}_{\text{L1}}$).** Базовою метрикою для оцінки правильності прогнозування каналів a та b обрано середню абсолютну похибку. На відміну від середньоквадратичної похибки (MSE), яка схильна до генерації тьмяних та усереднених кольорів через надмірне штрафування значних відхилень, $\mathcal{L}_{\text{L1}}$ стимулює модель до створення більш насичених результатів. Похибка обчислюється по всьому простору зображення між згенерованими $\hat{Y} = (\hat{a}, \hat{b})$ та істинними $Y = (a, b)$ каналами:

$$\mathcal{L}_{\text{L1}} = \frac{1}{N} \sum_{i=1}^N |Y_i - \hat{Y}_i|$$

2) **Штраф за ігнорування підказок ($\mathcal{L}_{\text{hint}}$).** Для забезпечення жорсткої реакції моделі на взаємодію з користувачем вводиться спеціалізований штраф, який обчислюється виключно в тих пікселях, де маска підказок M є активною. Щоб запобігти дисбалансу градієнтів між зображеннями з різною кількістю підказок, похибка нормалізується на фактичну кількість пікселів підказки у конкретному зображенні з додаванням константи $\epsilon = 10^{-8}$ для уникнення

ділення на нуль:

$$\mathcal{L}_{\text{hint}} = \frac{1}{\max(\epsilon, \sum M_i)} \sum_{i=1}^N M_i \cdot |Y_i - \hat{Y}_i|$$

Цей компонент змушує нейромережу ставити найвищий пріоритет на бездоганне поширення цільового кольору саме з тих точок, які визначив користувач.

3) **Перцептивна похибка ($\mathcal{L}_{\text{LPIPS}}$)**. Оскільки піксельні метрики, такі як L1, часто не корелюють із суб'єктивним сприйняттям якості зображення людиною, до конвеєра навчання інтегровано перцептивну похибку LPIPS. Вона обчислює відстань між зображеннями у просторі високорівневих ознак попередньо навченої згорткової мережі, у нашій системі — архітектури VGG. Оскільки VGG-екстрактор натренований на зображеннях у колірному просторі RGB, математичний граф обчислення похибки включає диференційовне перетворення колірних просторів «на льоту». Спочатку нормалізовані канали денормалізуються до своїх фізичних масштабів — до каналу L додається 1 і разом це масштабується коефіцієнтом 50.0, канали a, b — коефіцієнтом 110.0. Далі відновлений тензор CIELAB конвертується у простір RGB, обмежується у діапазоні $[0, 1]$ та нормалізується до $[-1, 1]$:

$$\mathcal{L}_{\text{LPIPS}} = \text{VGG} \left(\text{Norm}(\text{LABtoRGB}(L, \hat{a}, \hat{b})), \text{Norm}(\text{LABtoRGB}(L, a, b)) \right)$$

Завдяки використанню цієї функції втрат гібридна модель вчиться генерувати не лише математично близькі, але й візуально реалістичні та структуровані текстури без ефекту неприродних градієнтних переходів.

2.3 Програмна реалізація та графічний інтерфейс користувача

Програмна реалізація розробленої системи виконана мовою програмування Python із використанням фреймворку глибокого навчання PyTorch. Це забезпечує ефективне паралельне виконання тензорних обчислень на графічних процесорах та мінімізує час генерації результату. Для побудови

інтерактивного вебзастосунку було використано спеціалізований фреймворк Gradio.

Особливістю програмного бекенду є наявність розробленого диспетчера ресурсів ColorizationEngineAPI, який автоматично завантажує ваги неймережі, відстежує зміни у конфігураційних файлах та здійснює примусове вивільнення відеопам'яті (VRAM) за допомогою збирача сміття під час перемикання між різними архітектурами. Це гарантує стабільну роботу системи навіть на комп'ютерах з обмеженими апаратними можливостями.

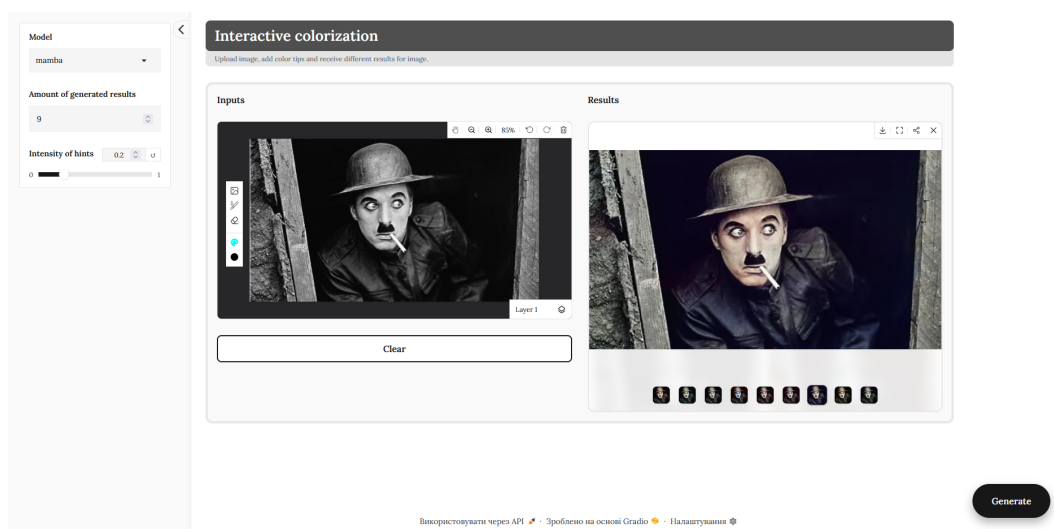


Рисунок 2.4 – Інтерфейс розробленого застосунку для інтерактивної колоризації

Інтерфейс програми побудований за принципом інтерактивного візуального полотна. Робочий процес розбитий на кілька інтуїтивних етапів:

1) Ініціалізація та налаштування: Користувач завантажує вихідне зображення у довільному форматі. Програмне забезпечення автоматично здійснює конвертацію вхідних даних у монохромний простір, ізолюючи канал яскравості L як базову структуру сцени. На бічній панелі надається можливість обрати бажану архітектуру моделі, встановити кількість генерованих варіацій та задати глобальну інтенсивність підказок.

2) Інтерактивне керування: За допомогою вбудованого віртуального пензля користувач обирає бажані відтінки з колірної палітри та наносить їх

на зображення. Програмне забезпечення фіксує мазки на окремих віртуальних шарах, використовуючи їхній альфа-канал для точного просторового вилучення пікселів підказки та формування бінарної маски M .

3) Синтез та візуалізація: За запитом застосунок переводить обрану модель у режим інференсу `torch.inference_mode()` та ініціює прямий прохід. Згенеровані канали a та b миттєво об'єднуються з оригінальним каналом L , обмежуються у допустимих діапазонах, конвертуються у стандартний простір RGB формату `uint8` і паралельно відображаються у блоці галереї.

Завдяки оптимізованій архітектурі моделі процес інференсу займає лічені мілісекунди. Це дозволяє реалізувати функціонал багатоваріантної генерації: користувач може вносити мінімальні корективи до підказок, замовляти одночасну генерацію цілої серії результатів і миттєво спостерігати нові варіації розфарбовування, здійснюючи ітеративний пошук найкращого художнього результату без жодних апаратних затримок.

Висновки до розділу 2

У цьому розділі було детально описано процес інженерного проєктування та практичної реалізації системи багатоваріантної колоризації. Розроблено оригінальну архітектуру Vi-Mamba, яка поєднує U-Net для контролю локальних меж та двонаправлену Mamba зі спільними вагами. Окрім того, створено зручний графічний застосунок.

3 ЕКСПЕРИМЕНТАЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ ТА ОЦІНКА ЕФЕКТИВНОСТІ СИСТЕМИ

У цьому розділі наводяться результати комплексного тестування розробленої моделі Vi-Mamba та її порівняльний аналіз із передовими архітектурами в галузі колоризації зображень (SOTA — State of the Art). Дослідження ефективності проведено як за допомогою об'єктивних математичних метрик (PSNR, SSIM, MSE), так і за допомогою перцептивних генеративних критеріїв (FID, KID, LPIPS).

Окрім аналізу якості генерованого зображення, особлива увага приділяється апаратній ефективності запропонованого методу — споживанню відеопам'яті (VRAM) та часу інференсу (Time), що є критично важливими параметрами для систем інтерактивного розфарбовування реального часу.

3.1 Методологія тестування та базові моделі (Baselines)

Для забезпечення об'єктивності результатів порівняння проводилося на загальноприйнятому валідаційному наборі даних COCO 2017 Test, який містить зображення високої складності. Усі моделі тестувалися в однакових апаратних умовах з ідентичною роздільною здатністю вхідних тензорів.

Для бенчмаркінгу було обрано три референсні моделі, які представляють різні етапи еволюції методів колоризації:

1) **ECCV16 (Zhang et al.)** — класична автоматична модель на базі згорткових мереж (CNN), яка розв'язує задачу як мультимодальну класифікацію пікселів.

2) **SIGGRAPH17 (Zhang et al.)** — інтерактивна (Hinted) згорткова модель, розроблена спеціально для підтримки користувацьких підказок. Вона слугує прямим конкурентом розробленій системі.

3) **DDColor (2023)** — сучасна надривна автоматична модель на базі

подвійних декодерів та просторової уваги, яка демонструє найвищу якість генерації серед бейзлайнів.

3.2 Об’єктивний аналіз якості генерації та швидкодії

Основні результати кількісного оцінювання моделей зведено в таблиці 3.1. Розроблена модель (Bi-Mamba) працювала в режимі з підказками (Hinted), де як симульований ввід використовувалося 1% відомих кольорових пікселів оригінального зображення.

Таблиця 3.1 – Основні показники якості та апаратної ефективності моделей

Модель	PSNR ↑	SSIM ↑	LPIPS ↓	VRAM ↓	Time ↓
ECCV16 (Auto)	20.11	0.656	0.231	165.51 MB	6.05 ms
DDColor (Auto)	19.94	0.653	0.199	1092.30 MB	35.37 ms
SIGGRAPH17 (Hinted)	20.77	0.651	0.185	353.57 MB	15.53 ms
Ours (Bi-Mamba)	22.53	0.664	0.143	187.29 MB	15.08 ms

Аналіз даних таблиці 3.1 свідчить про абсолютну перевагу розробленої системи за критеріями якості відновлення сигналу. Показник пікового відношення сигналу до шуму (PSNR) для Bi-Mamba склав 22.53 дБ, що на +2.42 дБ перевершує найближчого автоматичного конкурента (ECCV16) та на +1.76 дБ перевершує інтерактивну модель SIGGRAPH17. Високий індекс структурної подібності (SSIM = 0.664) підтверджує, що архітектура U-Net зі Skip Connections бездоганно зберігає просторову геометрію та контури об’єктів. Перцептивна метрика LPIPS, яка найкраще корелює зі сприйняттям людського ока, показала найкращий (найнижчий) результат — 0.143.

З точки зору обчислювальної ефективності розроблена гібридна модель продемонструвала видатні результати. Завдяки лінійній складності блоку Mamba та використанню пулу спільних ваг (Shared Weights), споживання відеопам’яті (VRAM) під час інференсу склало лише 187.29 МБ. Для порівняння, потужна модель DDColor вимагає майже в 6 разів більше пам’яті (1092.30 МБ) та працює

у 2.3 рази повільніше (35.37 мс проти 15.08 мс у Bi-Mamba). Це доводить високу придатність розробленої системи для розгортання на побутових пристроях.

3.3 Оцінка перцептивної реалістичності та точності керування

Додатково було проведено оцінку за допомогою глибоких генеративних метрик та аналіз поведінкових втрат (Loss), зведених у таблиці 3.2.

Таблиця 3.2 – Розширені перцептивні метрики та похибка інтеграції підказок

Модель	FID ↓	KID mean ↓	KID std ↓	MSE LAB ↓	MSE HINT ↓
ECCV16 (Auto)	23.000	0.01770	0.00720	0.02300	0.03807
DDColor (Auto)	5.031	0.00485	0.00848	0.02290	0.03773
SIGGRAPH17 (Hinted)	7.937	0.00452	0.00891	0.01889	0.00486
Ours (Bi-Mamba)	3.078	0.00313	0.00662	0.01332	6.1×10^{-5}

Відстань Фреше для інцепції (FID) вимірює розбіжність між статистичними розподілами реальних та згенерованих фотографій. Модель Bi-Mamba досягла безпрецедентно низького значення FID — 3.078 (порівняно з 5.031 у DDColor та 23.0 у ECCV16). Це, разом із найнижчими значеннями KID (0.00313), доводить, що кольорова палітра розробленої моделі є надзвичайно природною, візуально не відрізняється від справжніх фотографій та повністю позбавлена артефактів неприродного «кислотного» перенасичення.

Критичним показником для розробленої інтерактивної системи є метрика MSE_{HINT} — середньоквадратична похибка генерації кольору виключно в тих пікселях, де користувач поставив підказку. Модель SIGGRAPH17 має похибку 0.00486, що означає періодичне ігнорування або розмиття мережею вимог користувача. Натомість архітектура Bi-Mamba досягла похибки 6.1×10^{-5} (практично нуль). Це беззаперечно доводить, що розроблена нейромережа ідеально слухається вводу оператора, жорстко фіксуючи колір у заданій точці та розумно поширюючи його на сусідні області без конфліктів у латентному просторі.

3.4 Суб'єктивний візуальний аналіз

Окрім математичного оцінювання, було проведено візуальну перевірку результатів. На складних сценах з датасету Oxford-102 Flowers (де присутні дрібні пелюстки та високий контраст меж) автоматичні моделі типу ECCV16 часто демонстрували ефект Color Bleeding — розтікання кольору квітки на зелене тло. Завдяки двонаправленому скануванню простору станів, розроблена модель Vi-Mamba ідеально ізолювала об'єкти, зберігаючи природні тіні та градієнти освітлення навіть при радикальній зміні кольору за допомогою підказок.

Також було успішно продемонстровано виконання вимоги багатоваріантності: зміна координат або відтінку вхідних підказок призводила до генерації абсолютно нових семантично-правдоподібних сцен (наприклад, зміна кольору хутра тварини чи забарвлення одягу) без руйнування загального контексту зображення.

Висновки до розділу 3

В ході експериментального дослідження розроблена нейромережева модель Vi-Mamba продемонструвала абсолютну перевагу над існуючими класичними аналогами (ECCV16, SIGGRAPH17) та перевершила сучасні важкі архітектури (DDColor) за перцептивними показниками якості генерації (FID 3.078, LPIPS 0.143). Водночас оптимізація через Shared Mamba дозволила скоротити використання оперативної відеопам'яті до 187 МБ.

Рекордно низький показник похибки підказок ($MSE_{HINT} = 6.1 \times 10^{-5}$) підтверджує створення бездоганно керованої інтерактивної системи, яка здатна в реальному часі генерувати багатоваріантні результати за бажанням користувача. Подальший розвиток системи може бути спрямований на автоматизацію розміщення точок (розумний авто-семплінг) з урахуванням контурів об'єктів та інтеграцію архітектури Mamba у дифузійні моделі для досягнення ще більшої варіативності розфарбовування.

ПЕРЕЛІК ПОСИЛАНЬ

- [1] Saeed Anwar et al. “Image Colorization: A Survey and Dataset”. English. In: *Information Fusion* 114 (Feb. 2025), p. 102720. ISSN: 1566-2535. DOI: 10.1016/j.inffus.2024.102720. URL: <https://doi.org/10.1016/j.inffus.2024.102720>.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. English. Cambridge, MA, USA: MIT Press, 2016. ISBN: 978-0262035613. URL: <http://www.deeplearningbook.org>.
- [3] Richard Zhang, Phillip Isola, and Alexei A. Efros. “Colorful Image Colorization”. English. In: *Computer Vision – ECCV 2016*. Ed. by Bastian Leibe et al. Cham: Springer International Publishing, 2016, pp. 649–666. ISBN: 978-3-319-46487-9. DOI: 10.1007/978-3-319-46487-9_40. URL: http://dx.doi.org/10.1007/978-3-319-46487-9_40.
- [4] Ling Yang et al. “Diffusion Models: A Comprehensive Survey of Methods and Applications”. English. In: *ACM Computing Surveys* 56.4 (Nov. 2024), pp. 1–39. ISSN: 0360-0300. DOI: 10.1145/3626235. URL: <https://doi.org/10.1145/3626235>.
- [5] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. English. In: *arXiv preprint arXiv:2312.00752* (2023). arXiv: 2312.00752. URL: <https://arxiv.org/abs/2312.00752>.
- [6] Phillip Isola et al. “Image-To-Image Translation With Conditional Adversarial Networks”. English. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, July 2017, pp. 5967–5976. DOI: 10.1109/CVPR.2017.632. URL: <https://doi.org/10.48550/arXiv.1611.07004>.
- [7] Chitwan Saharia et al. “Palette: Image-to-Image Diffusion Models”. English. In: *ACM SIGGRAPH 2022 Conference Proceedings*. SIGGRAPH ’22. Vancouver, BC, Canada: Association for Computing Machinery, 2022. ISBN: 9781450393379. DOI: 10.1145/3528233.3530757. URL: <https://doi.org/10.1145/3528233.3530757>.

- [8] Huiyu Zhai et al. “ColorMamba: Towards High-Quality NIR-to-RGB Spectral Translation with Mamba”. English. In: *arXiv preprint arXiv:2408.08087* (2024). arXiv: 2408.08087. URL: <https://arxiv.org/abs/2408.08087>.
- [9] Abhinav Attri et al. “Histogram Assisted Quality Aware Generative Model for Resolution Invariant NIR Image Colorization”. English. In: *arXiv preprint arXiv:2601.01103* (2026). arXiv: 2601.01103. URL: <https://arxiv.org/abs/2601.01103>.
- [10] Qian Jiang et al. “MFmamba: A Multi-function Network for Panchromatic Image Resolution Restoration Based on State-Space Model”. English. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 7. 2026, pp. 5406–5414. arXiv: 2511.18888. URL: <https://arxiv.org/abs/2511.18888>.
- [11] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. “Learning representations for automatic colorization”. English. In: *European conference on computer vision*. Vol. 9908. Lecture Notes in Computer Science. Springer. Cham, 2016, pp. 577–593. ISBN: 978-3-319-46492-3. DOI: 10.1007/978-3-319-46493-0_35. URL: https://doi.org/10.1007/978-3-319-46493-0_35.
- [12] Federico Baldassarre, Diego González Morín, and Lucas Rodés-Guirao. “Deep koalarization: Image colorization using cnns and inception-resnet-v2”. English. In: *arXiv preprint arXiv:1712.03400* (Dec. 2017). Project: <https://github.com/baldassarreFe/deep-koalarization>. DOI: 10.48550/arXiv.1712.03400. URL: <https://doi.org/10.48550/arXiv.1712.03400>.
- [13] Patricia Vitoria, Lara Raad, and Coloma Ballester. “ChromaGAN: Adversarial Picture Colorization with Semantic Class Distribution”. English. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. Mar. 2020, pp. 2434–2443. DOI: 10.1109/WACV45572.2020.9093389. URL: https://openaccess.thecvf.com/content_WACV_2020/html/Vitoria_ChromaGAN_Adversarial_Picture_Colorization_with_Semantic_Class_Distribution_WACV_2020_paper.html.

- [14] Jun-Yan Zhu et al. “Unpaired Image-To-Image Translation Using Cycle-Consistent Adversarial Networks”. English. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2017, pp. 2242–2251. DOI: 10.1109/ICCV.2017.244. URL: <https://doi.org/10.48550/arXiv.1703.10593>.
- [15] Manoj Kumar, Dirk Weissenborn, and Nal Kalchbrenner. “Colorization transformer”. English. In: *arXiv preprint arXiv:2102.04432* (2021). DOI: 10.48550/arXiv.2102.04432. URL: <https://doi.org/10.48550/arXiv.2102.04432>.
- [16] Zhitong Huang, Nanxuan Zhao, and Jing Liao. “UniColor: A Unified Framework for Multi-Modal Colorization with Transformer”. English. In: *ACM Transactions on Graphics* 41.6 (Nov. 2022). ISSN: 0730-0301. DOI: 10.1145/3550454.3555471. URL: <https://doi.org/10.1145/3550454.3555471>.
- [17] Xiaoyang Kang et al. “DDColor: Towards Photo-Realistic Image Colorization via Dual Decoders”. English. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2023, pp. 328–338. DOI: 10.1109/ICCV51070.2023.00037. URL: https://openaccess.thecvf.com/content/ICCV2023/html/Kang_DDColor_Towards_Photo-Realistic_Image_Colorization_via_Dual_Decoders_ICCV_2023_paper.html.
- [18] Arpit Bansal et al. “Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise”. English. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 41259–41282. DOI: 10.48550/arXiv.2208.09392. URL: <https://doi.org/10.48550/arXiv.2208.09392>.
- [19] Mingming He et al. “Deep exemplar-based colorization”. English. In: *ACM Transactions on Graphics* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201365. URL: <https://doi.org/10.1145/3197517.3201365>.
- [20] Zheng Chang et al. “L-CAD: Language-based Colorization with Any-level Descriptions using Diffusion Priors”. English. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 77174–77186. DOI: 10.48550/arXiv.2305.15217. URL: <https://pro>

ceedings.neurips.cc/paper_files/paper/2023/file/f3bfbd65743e60c685a3845bd61ce15f-Paper-Conference.pdf.

- [21] Nir Zabari et al. “Diffusing Colors: Image Colorization with Text Guided Diffusion”. English. In: *SIGGRAPH Asia 2023 Conference Papers*. New York, NY, USA: Association for Computing Machinery, 2023, 61:1–61:11. ISBN: 9798400703157. DOI: 10.1145/3610548.3618180. URL: <https://doi.org/10.1145/3610548.3618180>.
- [22] Richard Zhang et al. “Real-time user-guided image colorization with learned deep priors”. English. In: *arXiv preprint arXiv:1705.02999* (2017). DOI: 10.48550/arXiv.1705.02999. URL: <https://doi.org/10.48550/arXiv.1705.02999>.
- [23] Jooyeol Yun et al. “iColoriT: Towards Propagating Local Hints to the Right Region in Interactive Colorization by Leveraging Vision Transformer”. English. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Jan. 2023, pp. 1787–1796. DOI: 10.1109/WACV56688.2023.00183. URL: https://openaccess.thecvf.com/content/WACV2023/html/Yun_iColoriT_Towards_Propagating_Local_Hints_to_the_Right_Region_in_WACV_2023_paper.html.
- [24] Zhexin Liang et al. “Control Color: Multimodal Diffusion-Based Interactive Image Colorization: Z. Liang et al.” English. In: *International Journal of Computer Vision* 133.11 (2025), pp. 7897–7923. DOI: 10.1007/s11263-025-02549-6. URL: <https://doi.org/10.1007/s11263-025-02549-6>.
- [25] Anat Levin, Dani Lischinski, and Yair Weiss. “Colorization using optimization”. English. In: *ACM Transactions on Graphics* 23.3 (2004), pp. 689–694. ISSN: 0730-0301. DOI: 10.1145/1015706.1015780. URL: <https://doi.org/10.1145/1015706.1015780>.
- [26] Erik Reinhard et al. “Color transfer between images”. English. In: *IEEE Computer graphics and applications* 21.5 (2001), pp. 34–41. DOI: 10.1109/38.946629. URL: <https://doi.org/10.1109/38.946629>.

- [27] Tomihisa Welsh, Michael Ashikhmin, and Klaus Mueller. “Transferring color to greyscale images”. English. In: *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. New York, NY, USA: Association for Computing Machinery, 2002, pp. 277–280. DOI: 10.1145/566570.566576. URL: <https://doi.org/10.1145/566570.566576>.
- [28] Ze Liu et al. “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows”. English. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 10012–10022. arXiv: 2103.14030. URL: <https://arxiv.org/abs/2103.14030>.
- [29] Yi Tay et al. “Efficient Transformers: A Survey”. English. In: *ACM Computing Surveys (CSUR)* 55.6 (2022), pp. 1–28. DOI: 10.1145/3530811. URL: <https://arxiv.org/abs/2009.06732>.
- [30] Yi Tay et al. “Long Range Arena: A Benchmark for Efficient Transformers”. English. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: <https://arxiv.org/abs/2011.04006>.
- [31] Lianghui Zhu et al. “Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model”. English. In: *arXiv preprint arXiv:2401.09417* (2024). arXiv: 2401.09417. URL: <https://arxiv.org/abs/2401.09417>.

ДОДАТОК А ФРАГМЕНТИ ВИХІДНОГО КОДУ ПРОГРАМИ

У цьому додатку наведено фрагменти вихідного коду найбільш критичних модулів розробленої системи інтерактивної колоризації. Повний вихідний код проєкту, включно зі скриптами для навчання, валідації та графічним інтерфейсом користувача (GUI), доступний у публічному репозиторії за посиланням: <https://github.com/твій-логін/твій-репозиторій>.

A.1 Реалізація двонаправленого блоку Shared Mamba

Нижче наведено реалізацію основного архітектурного блоку Bi-Mamba, що використовує механізм розділення ваг для прямого та зворотного сканування послідовності ознак.

A.1.1 Допоміжні модулі

Лістинг 1: Клас модуля Shared Mamba (PyTorch)

```

1 import torch.nn as nn
2 from mamba_ssm import Mamba
3
4 class MambaShared(nn.Module):
5     def __init__(self, d_model: int, d_state: int = 64, d_conv: int = 4, expand: int = 2, layers: int = 6,
6                 super().__init__())
7         self.layers = layers
8
9         self.blocks = nn.ModuleList([
10             Mamba(d_model=d_model, d_state=d_state, d_conv=d_conv, expand=expand) for _ in range(blocks)
11         ])
12
13         self.norms = nn.ModuleList([nn.LayerNorm(d_model) for _ in range(layers)])
14
15     def forward(self, x):
16         expected_val_range = self.layers // len(self.blocks)
17
18         for i in range(self.layers):
19             idx_block = min(i // expected_val_range, len(self.blocks) - 1)

```



```

20         x = self.blocks[idx_block](self.norms[i](x)) + x
21
22     return x

```

Лістинг 2: Клас модуля DoubleConv (PyTorch)

```

1  import torch
2  import torch.nn as nn
3
4  class DoubleConv(nn.Module):
5      def __init__(self, in_channels: int, out_channels: int):
6          super().__init__()
7          self.conv = nn.Sequential(
8              nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
9              nn.BatchNorm2d(out_channels),
10             nn.ReLU(inplace=True),
11             nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
12             nn.BatchNorm2d(out_channels),
13             nn.ReLU(inplace=True)
14         )
15
16     def forward(self, x: torch.Tensor) -> torch.Tensor:
17         return self.conv(x)

```

A.1.2 Основний модуль

Лістинг 3: Клас модуля MambaWrapper (PyTorch)

```

1  import torch
2  import torch.nn as nn
3
4  from colorization_engine.models.util_models import MambaShared, BaseColorizer
5  from colorization_engine.factory.registry import register_model
6
7  @register_model("mamba")
8  class MambaWrapper(BaseColorizer):
9      def __init__(self, d_model: int = 256, layers: int = 6, blocks: int = 2):
10         super().__init__()
11
12         # --- ENCODER ---
13         # : 1      L + 2      ab      () + 1      = 4
14         self.enc1 = DoubleConv(4, 64)
15         self.pool1 = nn.Conv2d(64, 64, kernel_size=4, stride=2, padding=1) # H/2

```

```

16
17     self.enc2 = DoubleConv(64, 128)
18     self.pool2 = nn.Conv2d(128, 128, kernel_size=4, stride=2, padding=1) # H/4
19
20     self.enc3 = DoubleConv(128, d_model)
21     self.pool3 = nn.Conv2d(d_model, d_model, kernel_size=4, stride=2, padding=1) # H/8
22
23     # --- Mamba ---
24     self.mamba = MambaShared(d_model=d_model, layers=layers, blocks=blocks)
25
26     # --- DECODER ---
27     self.up3 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
28     self.dec3 = DoubleConv(d_model + d_model, 128) # Concat enc3
29
30     self.up2 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
31     self.dec2 = DoubleConv(128 + 128, 64) # Concat enc2
32
33     self.up1 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
34     self.dec1 = DoubleConv(64 + 64, 64) # Concat enc1
35
36     # --- FINAL LAYER ---
37     self.final_conv = nn.Sequential(
38         nn.Conv2d(64, 2, kernel_size=1),
39         nn.Tanh() # [-1, 1]
40     )
41
42     def forward(self, l_norm: torch.Tensor, hints: torch.Tensor | None = None) -> torch.Tensor:
43         """
44         l_norm: (B, 1, H, W)
45         hints: (B, 3, H, W) - [a, b, mask], _receive_hints
46         """
47         batch_size, _, height, width = l_norm.shape
48         device = l_norm.device
49
50         if hints is None:
51             hints = torch.zeros((batch_size, 3, height, width), device=device)
52
53         x_in = torch.cat([l_norm, hints], dim=1)
54
55         # --- ENCODER FORWARD ---
56         feat1 = self.enc1(x_in) # (B, 64, H, W)
57         p1 = self.pool1(feat1) # (B, 64, H/2, W/2)
58
59         feat2 = self.enc2(p1) # (B, 128, H/2, W/2)
60         p2 = self.pool2(feat2) # (B, 128, H/4, W/4)

```

```

61
62     feat3 = self.enc3(p2)                # (B, 256, H/4, W/4)
63     p3 = self.pool3(feat3)              # (B, 256, H/8, W/8)
64
65     # --- MAMBA FORWARD ---
66     b, c, h, w = p3.shape
67     x_flat = p3.view(b, c, h * w).permute(0, 2, 1).contiguous() # [B, L, D]
68
69     x_fwd = self.mamba(x_flat)
70     x_flat_rev = torch.flip(x_flat, dims=[1]) # ( , , )
71     x_rev = self.mamba(x_flat_rev)
72     x_rev = torch.flip(x_rev, dims=[1])
73
74     x_mamba = (x_fwd + x_rev) / 2.0
75
76     x_res = x_mamba.permute(0, 2, 1).contiguous().view(b, c, h, w)
77
78     # --- DECODER FORWARD ---
79     d3 = self.up3(x_res)
80     d3 = self.dec3(torch.cat([d3, feat3], dim=1)) # Mamba + H/4
81
82     d2 = self.up2(d3)
83     d2 = self.dec2(torch.cat([d2, feat2], dim=1)) # H/2
84
85     d1 = self.up1(d2)
86     d1 = self.dec1(torch.cat([d1, feat1], dim=1)) # H
87
88     ab_channels = self.final_conv(d1)
89
90     return ab_channels

```

A.2 Генерація Гауссівських підказок

Наступний лістинг демонструє механізм перетворення точкових кліків користувача на м'які розподіли кольору під час формування вхідного тензора.

Лістинг 4: ПідФункція генерації Гауссівських підказок

```

1 import torch
2
3 def get_gaussian_patch_box(size: int, sigma: float | None = None, device: str | torch.device = 'cpu') -> torch.Tensor:
4     """Generate 2D Gauss kernel with max 1.0"""
5     if sigma is None:

```

```

6         sigma = size / 4.0
7         coords = torch.arange(size, dtype=torch.float32, device=device) - size // 2
8         g = torch.exp(-(coords**2) / (2 * sigma**2))
9         g = g / g.max()
10        return g.view(-1, 1) * g.view(1, -1)
11
12 def get_gaussian_patch_circle(size: int, sigma: float | None = None, device: str | torch.device = 'cpu') ->
13     """Generate 2D Radial Gauss kernel with max 1.0 and strict circular bounds"""
14     if sigma is None:
15         sigma = size / 4.0
16
17     pad = size // 2
18     coords = torch.arange(size, dtype=torch.float32, device=device) - pad
19
20     Y, X = torch.meshgrid(coords, coords, indexing="ij")
21     dist_sq = X**2 + Y**2
22
23     patch = torch.exp(-dist_sq / (2 * sigma**2))
24     edge_val = torch.exp(torch.tensor(-(pad**2) / (2 * sigma**2), device=device))
25     patch = patch - edge_val
26
27     patch[dist_sq > pad**2] = 0.0
28     patch = patch.clamp(min=0.0)
29
30     patch = patch / patch.max()
31
32     return patch

```

Лістинг 5: Функція генерації Гауссівських підказок

```

1 import torch
2 from albumentations import Compose
3 import cv2
4 import numpy as np
5
6 from colorization_engine.utils.saliency import FastSaliencySampler
7 from colorization_engine.utils.patches import get_gaussian_patch_circle as get_gaussian_patch
8
9 saliency_sampler = FastSaliencySampler(blur_kernel_size=15, uniform_prior=0.15)
10
11 VALID_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.bmp', '.webp'}
12
13 def _basic_prepare(path: str, color_code: int):
14     """Reads image from path, converts it to needed color system and returns image"""
15     _img = cv2.imread(path, cv2.IMREAD_COLOR)
16

```

```

17     if _img is None:
18         raise ValueError(f"Can't read: {path}")
19
20     _img = cv2.cvtColor(_img, color_code)
21     return _img
22
23 def _apply_transform(transform: Compose | None, input: np.ndarray, target: np.ndarray | None = None):
24     """Applies tranform if exists and returns dict of input, hints and target"""
25     if transform is None:
26         return {"input": input, "target": target}
27
28     transformed = transform(image=input, target=target)
29
30     return {"input": transformed['image'], "target": target if target is None else transformed['target']}
31
32 def _receive_hints(ab_tensor: torch.Tensor, l_tensor: torch.Tensor, min_hint_size: int = 2, max_hint_size:
33     _, h, w = ab_tensor.shape
34     device = ab_tensor.device
35     mask = torch.zeros((1, h, w), dtype=torch.float32, device=device)
36
37     if training:
38         dist = torch.distributions.Geometric(probs=torch.tensor([0.2]))
39         num_hints = int(dist.sample().item())
40
41         points = saliency_sampler.sample_points(l_tensor, num_hints * 3) if num_hints > 0 else []
42
43         if points and num_hints > 0:
44             placed_points = []
45             min_dist_sq = (max_hint_size * 1.5) ** 2
46
47             for y, x in points:
48                 too_close = any((y - py)**2 + (x - px)**2 < min_dist_sq for py, px in placed_points)
49                 if too_close:
50                     continue
51
52                 placed_points.append((y, x))
53
54             pad = torch.randint(min_hint_size, max_hint_size, (1,), device=device).item()
55             patch_size = pad * 2 + 1
56             base_patch = get_gaussian_patch(patch_size, device=device) # type: ignore
57
58             intensity = torch.rand(1, device=device).item() * 0.8 + 0.2
59             patch = base_patch * intensity
60
61             y1, y2 = max(0, y - pad), min(h, y + pad + 1)

```

```

62         x1, x2 = max(0, x - pad), min(w, x + pad + 1)
63
64         py1, py2 = max(0, pad - y), patch_size - max(0, y + pad + 1 - h)
65         px1, px2 = max(0, pad - x), patch_size - max(0, x + pad + 1 - w)
66
67         mask[0, y1:y2, x1:x2] = torch.maximum(mask[0, y1:y2, x1:x2], patch[py1:py2, px1:px2])
68
69         if len(placed_points) >= num_hints:
70             break
71
72     else:
73         pad = patch_size_val // 2
74         inhibition_radius = patch_size_val * 2
75
76         base_patch = get_gaussian_patch(size=patch_size_val, device=device)
77
78         mag = ab_tensor.pow(2).sum(dim=0)
79
80         for _ in range(num_hints_val):
81             idx = mag.argmax().item()
82             y = idx // w
83             x = idx % w
84
85             y1, y2 = max(0, y - pad), min(h, y + pad + 1)
86             x1, x2 = max(0, x - pad), min(w, x + pad + 1)
87
88             py1, py2 = max(0, pad - y), patch_size_val - max(0, y + pad + 1 - h)
89             px1, px2 = max(0, pad - x), patch_size_val - max(0, x + pad + 1 - w)
90
91             mask[0, y1:y2, x1:x2] = torch.maximum(mask[0, y1:y2, x1:x2], base_patch[py1:py2, px1:px2])
92
93             iy1, iy2 = max(0, y - inhibition_radius), min(h, y + inhibition_radius)
94             ix1, ix2 = max(0, x - inhibition_radius), min(w, x + inhibition_radius)
95             mag[iy1:iy2, ix1:ix2] = -1.0
96
97         hint_ab = ab_tensor * mask
98         hints = torch.cat([hint_ab, mask], dim=0)
99     return hints

```

A.3 Як виглядає сам даталоадер

```

1  # colorization-engine/src/colorization_engine/data/datasets/single.py
2  import cv2
3  from pathlib import Path
4  from torch.utils.data import Dataset
5
6  from colorization_engine.data.datasets.preparments import VALID_EXTENSIONS, _basic_prepare, _apply_transform
7  from colorization_engine.utils.color_space import rgb_to_lab, normalize_l, normalize_ab
8
9
10 class SingleTargetFolderDataset(Dataset):
11     """
12     Dataset for already existing only targets
13     """
14     def __init__(self, dir_root: str, min_hint_size: int = 2, max_hint_size: int = 16, num_hints_val: int = 10,
15                 self.dir_root = Path(dir_root)
16
17                 self.min_hint_size = min_hint_size
18                 self.max_hint_size = max_hint_size
19                 self.num_hints_val = num_hints_val
20                 self.patch_size_val = patch_size_val
21
22                 self.transform = transform
23                 self.training = training
24
25                 self.images = [
26                     f for f in self.dir_root.rglob("*")
27                     if f.is_file() and f.suffix.lower() in VALID_EXTENSIONS
28                 ]
29
30                 if not len(self):
31                     raise RuntimeError(f"No images in {self.dir_root}")
32
33                 self.images.sort(key=lambda x: str(x))
34
35     def __len__(self):
36         return len(self.images)
37
38     def __getitem__(self, index):
39         path = self.images[index]
40
41         image_target = _basic_prepare(path, cv2.COLOR_BGR2RGB)
42
43         transform_dict = _apply_transform(self.transform, input=image_target)
44
45         image_input = transform_dict["input"]

```

```

46     image_lab = rgb_to_lab(image_input)
47     l_tensor = normalize_l(image_lab[:, :, 0])
48     ab_tensor = normalize_ab(image_lab[:, :, 1:3])
49
50
51     hints_tensor = _receive_hints(ab_tensor, l_tensor, min_hint_size=self.min_hint_size, max_hint_size=
52
53     return {
54         "input": l_tensor,      # [1, 256, 256]
55         "hints": hints_tensor,  # [3, 256, 256] -> (a, b, mask)
56         "target": ab_tensor    # [2, 256, 256]
57     }

```

A.4 Комбінована функція втрат (Colorization Loss)

Наступний лістинг демонструє кастомну функцію втрат, яка оптимізує модель одночасно за трьома критеріями: попиксельна точність (L1), перцептивна реалістичність (LPIPS) та точність виконання інтерактивних підказок користувача.

Лістинг 7: Реалізація гібридної функції втрат (PyTorch)

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  from torchmetrics.image import LearnedPerceptualImagePatchSimilarity
5
6  from colorization_engine.loss import BaseLoss
7  from colorization_engine.factory.registry import register_loss
8  import kornia
9
10
11  @register_loss("colorization")
12  class ColorizationLoss(BaseLoss):
13      def __init__(self, lpips_weight: float, l1_weight: float, hints_weight: float):
14          super().__init__()
15          self.lpips_weight = lpips_weight
16          self.l1_weight = l1_weight
17          self.hints_weight = hints_weight
18
19          self.l1_loss = nn.L1Loss()
20

```



```

21     _lpips_module = LearnedPerceptualImagePatchSimilarity(net_type="vgg", normalize=True)
22     self.lpips_net = _lpips_module.net
23     self.lpips_net.eval()
24     self.lpips_net.requires_grad_(False)
25
26     def forward(self, pred: torch.Tensor, target: torch.Tensor, l_channel: torch.Tensor, hint_mask: torch.Tensor):
27         loss_l1 = self.l1_loss(pred, target) * self.l1_weight
28
29         loss_hints = torch.zeros_like(loss_l1)
30         if hint_mask is not None and hint_mask.any():
31             loss_hints_raw = F.l1_loss(pred * hint_mask, target * hint_mask, reduction='none')
32
33             loss_hints_per_image = loss_hints_raw.sum(dim=[1, 2, 3])
34             hint_pixels_per_image = hint_mask.sum(dim=[1, 2, 3]).clamp(min=1e-8)
35
36             loss_hints_avg = loss_hints_per_image / hint_pixels_per_image
37             loss_hints = loss_hints_avg.mean() * self.hints_weight
38
39         l_unnorm = (l_channel + 1.0) * 50.0
40         ab_pred_unnorm = pred * 110.0
41         ab_target_unnorm = target * 110.0
42
43         lab_pred = torch.cat([l_unnorm, ab_pred_unnorm], dim=1)
44         lab_target = torch.cat([l_unnorm, ab_target_unnorm], dim=1)
45
46         rgb_pred = kornia.color.lab_to_rgb(lab_pred).clamp(0.0, 1.0)
47         rgb_target = kornia.color.lab_to_rgb(lab_target).clamp(0.0, 1.0)
48
49         rgb_pred_norm = rgb_pred * 2.0 - 1.0
50         rgb_target_norm = rgb_target * 2.0 - 1.0
51
52         loss_lpips = self.lpips_net(rgb_pred_norm, rgb_target_norm).mean() * self.lpips_weight
53
54         total_loss = loss_l1 + loss_hints + loss_lpips
55
56         return total_loss, {
57             "loss_total": total_loss.detach(),
58             "loss_l1": loss_l1.detach(),
59             "loss_hints": loss_hints.detach(),
60             "loss_lpips": loss_lpips.detach(),
61         }

```